

techwave.unipd@gmail.com

Specifica tecnica - v1.0.0

Dettagli documento:

Classificazione: Esterno

Versione: 1.0.0

List of Figures	3
1 Introduzione	5
1.1 Scopo del documento	5
1.2 Scopo del prodotto	5
1.3 Glossario	5
1.4 Riferimenti	5
1.4.1 Riferimenti normativi	5
1.4.2 Riferimenti informativi	5
2 Tecnologie	6
2.1 Linguaggio di programmazione	6
2.1.1 TypeScript	6
2.2 Frameworks	6
2.2.1 VSCode Extension API	6
2.3 Strumenti per l'integrazione con modelli di intelligenza artificiale	7
2.3.1 Ollama	7
2.3.2 Vector Embeddings	7
2.3.3 LanceDB	8
2.4 Strumenti per i test	8
2.4.1 Jest	8
2.5 Formati di file per la gestione dei requisiti	9
2.5.1 CSV	9
2.5.2 ReqIF	9
3 Architettura	9
3.1 Architettura logica	9
3.1.1 Eventi VSCode	9
3.1.2 Model View Presenter (MVP)	9
3.2 Architettura di deployment	10
3.2.1 Monolite	10
3.3 Design pattern	11
3.3.1 Dependency Injection	11
3.3.2 MVP	11
3.3.3 Singleton	11
3.3.4 Adapter	11
3.3.5 Facade	11
3.3.6 Command	12
3.3.7 Memento	12
3.4 Diagramma delle classi	12
3.4.1 Architettura di dettaglio	13
3.4.2 Model	13
3.4.3 View	15
3.4.4 Presenter	15
3.4.5 Command	17
3.4.6 Servizi e facade	18
3.5 Progettazione grafica	26

3.5.1	Import	27
3.5.2	Track	27
3.5.3	Results	28
3.5.4	Chat	28

List of Figures

1	UML Diagram	13
2	Models UML Diagram	13
3	Views UML Diagram	15
4	Providers UML Diagram	15
5	Commands UML Diagram	17
6	FileSystem Service UML Diagram	18
7	GlobalState Service UML Diagram	19
8	Config Service UML Diagram	19
9	Requirement Service UML Diagram	20
10	Tracking Result Service UML Diagram	22
11	Document Service UML Diagram	23
12	Inference Service UML Diagram	24
13	Chat Service UML Diagram	26
14	Pannello Import	27
15	Pannello Track	27
16	Pannello Results	28

Table 1: Changelog

Data	Versione	Descrizione	Autore	Data Verifica	Verificatore
12/04/2025	1.0.0	Sezione Progettazione grafica	Carraro Agnese	14/04/2025	Marcon Giulia
08/04/2025	0.6.0	Aggiunta classi View e Command	Piola Andrea	09/04/2025	Dal Bianco Riccardo
08/04/2025	0.5.1	Correzione sezione Tecnologie	Marcon Giulia	08/04/2025	Pistori Gaia
02/04/2025	0.5.0	Sezione Diagramma delle classi	Dal Bianco Riccardo	06/04/2025	Monetti Luca
28/03/2025	0.4.0	Sezione Design pattern	Piola Andrea	30/03/2025	Monetti Luca
25/03/2025	0.3.0	Sezione Architettura logica	Pistori Gaia	25/03/2025	Piola Andrea
19/03/2025	0.2.0	Sezione Tecnologie	Marcon Giulia	21/03/2025	Monetti Luca
18/03/2025	0.1.0	Prima stesura del documento. Sezione Introduzione	Vasquez Manuel Felipe	19/03/2025	Marcon Giulia

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione dettagliata del sistema, delle sue *funzionalità*_[G] e dei requisiti tecnici necessari per la sua realizzazione.

1.2 Scopo del prodotto

Nello sviluppo di software per sistemi embedded, il controllo dell'implementazione di tutti i requisiti necessari al corretto funzionamento del sistema risulta costoso e ripetitivo per lo sviluppatore, oltre a poter essere non esaustivo a causa di distrazioni o dimenticanze. Il *capitolato*_[G] **Requirement Tracker - Plug-in VSCode** propone lo sviluppo di un *plugin*_[G] per *VSCode*_[G] che permetta di tracciare i requisiti derivanti da documenti tecnici di sistemi embedded, valutare se il codice del software scritto da sviluppatori implementi i vari requisiti in modo esaustivo, ed in caso di mancata implementazione dia un avviso per avvertire dell'effettiva assenza.

1.3 Glossario

Per evitare incomprensioni riguardanti la terminologia utilizzata all'interno dei vari documenti, viene fornito un Glossario che racchiude tutti i vari termini tecnici, potenzialmente ambigui, con la propria definizione precisa. I termini presenti nel glossario saranno evidenziati nei documenti nei seguenti modi:

- **Sito Web:** Grassetto colorato.
- **PDF:** Corsivo con pendice [G].

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Norme di Progetto:** Norme di Progetto - v1.8.1
- **Capitolato d'appalto C8: Requirement Tracker- Plug-in VSCode:** <https://www.math.unipd.it/~tullio/IS-1/2024/Progetto/C8.pdf>

1.4.2 Riferimenti informativi

- **Glossario:** Glossario
- **Analisi dei requisiti:** Analisi dei requisiti
- **Documentazione dell'Extension API di Visual Studio Code:** <https://code.visualstudio.com/api>
- **Documentazione di Ollama:** <https://github.com/ollama/ollama/tree/main/docs>
- **Documentazione di TypeScript:** <https://www.typescriptlang.org/docs/>
- **Documentazione di LanceDB:** <https://docs.lancedb.com/enterprise/introduction>

2 Tecnologie

In questa sezione vengono descritti gli strumenti e le tecnologie utilizzati per lo sviluppo e l'implementazione del Requirement Tracker, un'estensione per Visual Studio Code ($VSCode_{[G]}$) che permette di tracciare e verificare i requisiti software. Le tecnologie sono state selezionate per garantire un'*architettura* $_{[G]}$ modulare, scalabile e di facile manutenzione, con un focus particolare sull'integrazione con modelli di linguaggio avanzati ($LLM_{[G]}$) e sulla gestione efficiente dei requisiti.

2.1 Linguaggio di programmazione

2.1.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft, che estende JavaScript aggiungendo il supporto per la tipizzazione statica. Questo permette di identificare errori comuni durante la fase di sviluppo, migliorando la *qualità* $_{[G]}$ del codice e facilitando il debugging. TypeScript è particolarmente adatto per progetti di medie e grandi dimensioni, dove la manutenibilità del codice è fondamentale. È stato scelto per la sua capacità di prevenire errori attraverso la tipizzazione statica, la vasta gamma di strumenti e librerie disponibili e la facilità di integrazione con le API di $VSCode_{[G]}$.

Utilizzo nel progetto:

- Sviluppo dell'estensione per $VSCode_{[G]}$: TypeScript è utilizzato per implementare le *funzionalità* $_{[G]}$ principali dell'estensione, come il caricamento dei requisiti, l'analisi del codice e l'interazione con l'utente.
- Gestione delle *funzionalità* $_{[G]}$ principali: Il linguaggio è impiegato per gestire la logica di business, come il parsing dei file di requisiti, l'interazione con i modelli $LLM_{[G]}$ e la gestione della struttura dati interna.
- Integrazione con le API di $VSCode_{[G]}$: TypeScript è utilizzato per interfacciarsi con le API di $VSCode_{[G]}$, permettendo la creazione di comandi personalizzati, la visualizzazione dei risultati e l'integrazione con l'editor di codice.

Versione: 4.9.5

Documentazione: [TypeScript documentation](#)

2.2 Frameworks

2.2.1 VSCode Extension API

Le API di estensione di Visual Studio Code forniscono strumenti per la creazione di comandi, la gestione dei file, l'integrazione con l'editor di codice e la visualizzazione dei risultati. Queste API sono progettate specificamente per estendere le *funzionalità* $_{[G]}$ di $VSCode_{[G]}$, permettendo agli sviluppatori di creare strumenti personalizzati che si integrano perfettamente con l'ambiente di sviluppo. Sono state scelte per la loro integrazione nativa con $VSCode_{[G]}$, che semplifica lo sviluppo di estensioni e garantisce una perfetta compatibilità con l'ambiente di sviluppo.

Utilizzo nel progetto:

- Presentazione dell'interfaccia utente dell'estensione: Le API di $VSCode_{[G]}$ sono utilizzate per selezionare il provider dell'interfaccia utente dell'estensione che si occuperà della gestione degli eventi e del render del codice HTML.

- Integrazione con l'editor di *VSCode_[G]*: Le API permettono di interagire con l'editor di codice, ad esempio per evidenziare porzioni di codice che implementano specifici requisiti o per navigare tra i file del *progetto_[G]*.
- Gestione degli eventi: Le API di estensione sono utilizzate per gestire eventi come il caricamento di un file di requisiti, l'avvio di un'analisi o la modifica delle configurazioni.

Versione: 1.95.0

Documentazione: VSCode API documentation

2.3 Strumenti per l'integrazione con modelli di intelligenza artificiale

2.3.1 Ollama

Ollama_[G] è uno strumento open-source che permette di eseguire localmente modelli di linguaggio avanzati (*LLM_[G]*) come *Llama_[G]* ed è progettato per essere leggero e facile da configurare, permettendo agli sviluppatori di eseguire modelli *LLM_[G]* direttamente sulla propria macchina senza la necessità di infrastrutture cloud complesse. Uno dei principali vantaggi di *Ollama_[G]* è la sua capacità di eseguire modelli *LLM_[G]* in locale, garantendo la privacy dei dati e riducendo la dipendenza da servizi esterni. Inoltre, *Ollama_[G]* supporta una vasta gamma di modelli, permettendo di scegliere quello più adatto alle esigenze del *progetto_[G]*.

Utilizzo nel progetto:

- Analisi del codice sorgente: *Ollama_[G]* è utilizzato per analizzare il codice sorgente e verificare l'implementazione dei requisiti. Il modello *LLM_[G]* viene interrogato per generare risposte basate sul codice e sui requisiti specificati.
- Generazione di risposte basate sui modelli *LLM_[G]*: *Ollama_[G]* è utilizzato per generare risposte che supportano lo sviluppatore nel controllo dei requisiti, ad esempio identificando porzioni di codice che implementano specifici requisiti.
- Generazione degli embeddings: *Ollama_[G]* è utilizzato per generare gli embeddings vettoriali del codice sorgente e dei requisiti.

Versione: 0.6.2

Documentazione: Ollama documentation

2.3.2 Vector Embeddings

I Vector Embeddings sono rappresentazioni numeriche di dati testuali, utilizzati per trasformare requisiti e codice in vettori. Questa tecnica permette di confrontare rapidamente i requisiti con il codice sorgente, migliorando l'*efficienza_[G]* dell'analisi e la precisione dei risultati. I Vector Embeddings sono particolarmente utili in contesti dove è necessario confrontare grandi quantità di dati testuali, come nel caso dei requisiti software. I Vector Embeddings funzionano mappando parole, frasi o interi documenti in uno spazio vettoriale multidimensionale, dove la similarità semantica tra i testi può essere misurata attraverso la distanza tra i vettori. Questo approccio permette di identificare rapidamente le corrispondenze tra i requisiti e il codice sorgente, riducendo i tempi di elaborazione e migliorando l'accuratezza dell'analisi.

Utilizzo nel progetto:

- Confronto rapido tra i requisiti e il codice sorgente: I Vector Embeddings sono utilizzati per trasformare i requisiti e il codice sorgente in vettori, permettendo di confrontarli rapidamente e identificare le corrispondenze.

- Miglioramento dell'*efficienza_[G]* e della precisione dell'analisi: L'uso di Vector Embeddings permette di ridurre i tempi di elaborazione e migliorare l'accuratezza delle risposte fornite dai modelli *LLM_[G]*.

Documentazione: Vector Embeddings

2.3.3 LanceDB

LanceDB_[G] è un database vettoriale open-source progettato per offrire elevate prestazioni nella gestione e nella ricerca di Vector Embeddings. È particolarmente indicato per applicazioni che fanno uso intensivo di modelli di linguaggio (*LLM_[G]*) e analisi semantica di dati testuali. LanceDB combina un motore di ricerca vettoriale efficiente con un formato di storage ottimizzato (Lance), permettendo interrogazioni rapide anche su dataset di grandi dimensioni.

Utilizzo nel progetto:

- Indicizzazione e ricerca vettoriale: *LanceDB_[G]* viene utilizzato per indicizzare i requisiti e il codice sorgente rappresentati come embeddings vettoriali, permettendo un confronto efficiente basato sulla similarità semantica.
- Persistenza locale dei dati: Il database permette di salvare localmente i dati elaborati, garantendo prestazioni elevate senza dover dipendere da servizi cloud.
- Ottimizzazione delle risposte *LLM_[G]*: Utilizzando *LanceDB_[G]* è possibile migliorare la qualità delle risposte generate dai modelli *LLM_[G]*, restringendo il contesto ai risultati più rilevanti trovati tramite la ricerca vettoriale.

Versione: 0.18

Documentazione: LanceDB documentation

2.4 Strumenti per i test

2.4.1 Jest

Jest è un framework di testing JavaScript open-source sviluppato da Facebook. È utilizzato per scrivere e eseguire *test_[G]* automatizzati, garantendo che il codice funzioni come previsto. Jest è particolarmente apprezzato per la sua configurazione minima e per le sue capacità avanzate come il mock delle funzioni e la gestione dei *test_[G]* paralleli. È compatibile con progetti che utilizzano TypeScript e offre un'ottima integrazione con altri strumenti di sviluppo, come i framework di build e i bundler.

Utilizzo nel progetto:

- Testing unitario: Jest è utilizzato per sviluppare *test_[G]* unitari che verificano il corretto funzionamento delle singole funzioni e componenti dell'estensione.
- Testing di integrazione: I *test_[G]* di integrazione sono stati implementati per garantire che diverse parti dell'estensione interagiscano correttamente, ad esempio, tra la logica di business e l'integrazione con le API di *VSCoDe_[G]*.
- Mocking di funzioni esterne: Jest consente di simulare comportamenti di funzioni esterne come la comunicazione con i modelli di linguaggio, facilitando i *test_[G]* senza dipendenze esterne.

Versione: 29.7.0

Documentazione: Jest documentation

2.5 Formati di file per la gestione dei requisiti

2.5.1 CSV

Il formato CSV (Comma-Separated Values) è un formato di file semplice e leggero, ampiamente utilizzato per la gestione di dati strutturati. Consente di importare ed esportare i requisiti software in modo rapido e intuitivo. CSV è particolarmente adatto per progetti che richiedono una gestione semplice e veloce dei dati, senza la necessità di strutture complesse.

Utilizzo nel progetto:

- Importazione dei requisiti software: CSV è utilizzato per importare i requisiti software, permettendo una facile integrazione con strumenti esterni per la gestione dei dati.

Documentazione: CSV Format specification

2.5.2 ReqIF

ReqIF (Requirements Interchange Format) è uno standard per lo scambio di requisiti tra diversi strumenti di gestione. Offre un formato strutturato e flessibile, adatto a progetti complessi che richiedono una gestione avanzata dei requisiti. ReqIF è progettato per supportare la gestione di grandi quantità di requisiti, garantendo la compatibilità tra diversi strumenti e piattaforme.

Utilizzo nel progetto:

- Importazione dei requisiti software: ReqIF è utilizzato per importare i requisiti software, permettendo una facile integrazione con strumenti esterni per la gestione dei dati.

Documentazione: ReqIF specification

3 Architettura

3.1 Architettura logica

L'*architettura_[G]* implementata utilizza il pattern Model View Presenter per gestire l'interazione con l'utente. L'interazione tra presenter e view avviene tramite gli eventi di *VSCode_[G]* mentre l'interazione tra presenter e model avviene per chiamata diretta.

3.1.1 Eventi VSCode

L'estensione si attiva tramite l'evento nativo fornito dall'API. La comunicazione tra un provider e la sua view avviene tramite l'ascolto e la gestione di eventi personalizzati e specificati negli handler.

3.1.2 Model View Presenter (MVP)

Il Model-View-Presente è una derivazione dello schema Model-View-Controller (MVC). Entrambi sono ampiamente utilizzati per la creazione di applicazioni con interfaccia utente. I principali vantaggi del modello MVP sono:

- Separation of Concerns: dividere il codice in parti separate, ciascuna con la propria responsabilità. Ciò rende il codice più semplice, più riutilizzabile e più facile da gestire.

- Unit Testing: poiché la logica (il presenter) dell'interfaccia utente è separata dal livello visivo (la view), è molto più facile testare queste parti in modo isolato.

In MVP le tre componenti sono suddivise come segue:

- Model: Livello per la gestione dei dati. È responsabile dell'implementazione della business logic e della comunicazione con il database.
- View: Livello per visualizzazione dell'interfaccia utente. Visualizza i dati (dal model) e indirizza i comandi dell'utente (eventi) al presenter affinché agisca su tali dati.
- Presenter: Livello che permette l'interazione tra Model e View. Recupera i dati dal model e decide cosa visualizzare. Gestisce lo stato della vista e intraprende azioni in base alle notifiche di input dell'utente dalla vista.

3.2 Architettura di deployment

3.2.1 Monolite

Un'*architettura_[G]* monolitica è un modello tradizionale di un programma software, che è costruito come un'unità unificata, autosufficiente e indipendente da altre applicazioni. Un'*architettura_[G]* monolitica è una rete di elaborazione singolare e di grandi dimensioni con una codebase che unisce insieme tutti i business concerns. Per apportare una modifica a questo tipo di applicazione è necessario aggiornare l'intero stack accedendo alla codebase e creando e distribuendo una versione aggiornata dell'interfaccia lato servizio. Ciò rende gli aggiornamenti restrittivi, dispendiosi in termini di tempo e consente di rilasciare tutto il monolite in una volta.

I vantaggi di un'*architettura_[G]* monolitica includono:

- Facile distribuzione : un file eseguibile o una directory semplificano la distribuzione.
- Sviluppo: l'utilizzo di una singola codebase rende più lineare lo sviluppo dell'applicativo.
- Prestazioni: in una codebase e un *repository_[G]* centralizzati, un'API può spesso eseguire la stessa funzione che numerose API eseguono con i microservizi.
- *Test_[G]* semplificati: poiché un'applicazione monolitica è un'unità singola e centralizzata, i *test_[G]* end-to-end possono essere eseguiti più velocemente rispetto ad un'applicazione distribuita.
- Debug semplice: con tutto il codice in un unico posto, è più facile seguire una richiesta e trovare un problema.

Gli svantaggi di un monolite includono:

- Velocità di sviluppo più lenta: un'applicazione monolitica e di grandi dimensioni rende lo sviluppo più complesso e lento.
- Scalabilità: non è possibile scalare i singoli componenti.
- Affidabilità: se si verifica un errore in un modulo, ciò potrebbe influire sulla disponibilità dell'intera applicazione.
- Barriera all'adozione della tecnologia: qualsiasi modifica al framework o al linguaggio influisce sull'intera applicazione, rendendo spesso le modifiche costose e dispendiose in termini di tempo.
- Mancanza di flessibilità: un monolite è vincolato dalle tecnologie già utilizzate al suo interno.
- Distribuzione: una piccola modifica a un'applicazione monolitica richiede la ridistribuzione dell'intero monolite.

3.3 Design pattern

3.3.1 Dependency Injection

È un *pattern architetturale* che consiste nel fornire le dipendenze di un oggetto dall'esterno, tramite il passaggio di parametri nel costruttore, invece di crearle internamente. Questo facilita l'implementazione di *test_[G]* di unità che utilizzano mock e garantisce di avere oggetti validi sin dall'istanziamento dell'oggetto della classe.

3.3.2 MVP

È un *pattern architetturale* con lo scopo di separare le responsabilità dei componenti di un'applicazione. Come già indicato nella sezione *Architettura Logica* è stato utilizzato per l'implementazione dell'intero *applicativo_[G]* in quanto era necessario coordinare le componenti di dati e logica e interfaccia utente. Il *model* è implementata dalle classi *Facade* e *Service*. La *view*, ovvero l'interfaccia grafica, è implementata dalla classe *TrackerWebView*, che rappresenta il pannello di visualizzazione dello stato di implementazione dei requisiti, e dalla classe *ChatWebView* che rappresenta la chat tra l'utente e il modello. Il *presenter* è implementato dalle rispettive classi *ChatWebviewProvider* e *TrackerWebviewProvider* e rappresenta la comunicazione tra la business logic e l'interfaccia utente.

3.3.3 Singleton

È un *pattern creazionale* che garantisce l'esistenza di un'unica istanza di una classe e permette di avere un punto di accesso globale a quest'ultima. È utilizzato nella classe *ConfigServiceFacade* rappresenta le configurazioni scelte dall'utente sia nel contesto globale che nel singolo progetto ed è necessario quindi siano memorizzate in un'unica istanza per non generare conflitti. È utilizzato anche nella classe *LanceDBAdapter* che si occupa dell'interazione con il database vettoriale. In entrambi i casi risulta molto utile l'uso del Singleton per avere un punto di accesso globale alle informazioni.

3.3.4 Adapter

È un *pattern strutturale* che permette di convertire un'interfaccia di una classe in un'altra. Questo si implementa definendo una classe adapter che adatti le interfacce. È stato utilizzato nella classe *LanceDBAdapter* per permettere all'*applicativo_[G]* di interfacciarsi con il database vettoriale per memorizzare il parsing dei requisiti e il loro stato. È stato utilizzato anche nella *LangChainOllamaAdapter* che permette all'*applicativo_[G]* di interfacciarsi con *Ollama_[G]* per gestire l'interrogazione e il funzionamento dei modelli *LLM_[G]*.

3.3.5 Facade

È un *pattern strutturale* che permette di fornire un'interfaccia unica semplice per un sottosistema complesso. Questo permette di diminuire la complessità del sistema. È stato usato nella classe *DocumentServiceFacade* la quale fornisce una serie di metodi per la formattazione e la trasformazione in formato vettoriale dei documenti da analizzare da *LLM_[G]*. La classe richiede il contesto dell'estensione per analizzare i file selezionati e per permetterne l'elaborazione e la memorizzazione nel database. In modo simile, è stato usato nella classe *RequirementsServiceFacade* la quale implementa al suo interno i metodi per la gestione dell'importazione e il tracciamento dei requisiti. È stato usato anche nella classe *ConfigServiceFacade* la quale mette a disposizione i metodi per interagire con le configurazioni dell'utente. Queste scelte hanno

permesso di diminuire la complessità del sistema unificando funzionalità articolate all'interno di metodi di utilità.

3.3.6 Command

È un *pattern comportamentale* che permette di incapsulare una richiesta in un oggetto con lo scopo di rendere i client indipendenti dalle richieste. Una classe astratta, *Command*, definisce l'interfaccia per eseguire la richiesta. È stata creata un'interfaccia *ICommand*, contenente una *Promise* la quale viene implementata dalle classi: *ClearChatHistoryCommand*, *ClearRequirementsHistoryCommand*, *InterrogateDocumentCommand*, *InterrogateSelectionCommand*, *OpenSettingsCommand*, *OpenSidebarCommand* e *ResetDatabaseCommand*. Queste corrispondono ai comandi che l'utente può richiedere all'estensione. È stato scelto questo pattern per garantire una facile estensione delle funzionalità, rendendo semplice l'aggiunta di nuovi comandi, e per garantire il rispetto del *principio di separazione delle responsabilità*.

3.3.7 Memento

È un *pattern comportamentale* che permette di salvare e recuperare lo stato di un oggetto senza rivelare dettagli della sua implementazione. Questa scelta architetturale è imposta da *VSCode_[G]* Extension API che richiede di utilizzarlo per la gestione del *workspaceState* e del *globalState*, da noi utilizzate nella classe *GlobalStateService*.

Documentazione Memento

3.4 Diagramma delle classi

Il diagramma delle classi fornisce una panoramica generale della struttura interna dell'estensione, evidenziando i principali componenti software, le loro responsabilità e le relazioni tra essi. L'*architettura_[G]* è progettata per essere modulare ed estendibile usando il pattern MVP.

I componenti sono suddivisi in diversi blocchi funzionali:

- **Command:** Rappresentano le azioni attivabili dall'utente. Sono centralizzati nel *CommandRegistry* e implementano un'interfaccia comune che definisce il comportamento standard dei comandi.
- **Interfaccia VSCode:** Contiene le componenti fornite da *VSCode_[G]* e gestisce la creazione e la visualizzazione delle *WebView*.
- **Provider:** Agiscono come intermediari tra la *WebView* e i servizi applicativi. Ricevono eventi dalla UI e instradano le richieste verso i servizi appropriati.
- **WebView:** Sono i componenti grafici visualizzati all'interno dell'estensione. Si interfacciano con i rispettivi provider per ricevere dati o inviare eventi tramite *VSCode_[G]*.
- **Interazioni con servizi esterni:** Contengono la logica principale per la gestione delle interazioni con *Ollama_[G]* e *LanceDB_[G]*. Interagiscono direttamente con gli adapter.
- **Gestione dello stato:** Mantiene lo stato persistente dell'estensione e fornisce metodi di lettura e scrittura.
- **Adapter:** Permettono di astrarre e integrare tecnologie esterne per il database vettoriale e i modelli *LLM_[G]*.
- **Servizi di tracciamento e gestione requisiti:** Coordinano le attività legate alla gestione, analisi e tracciamento dei requisiti software.
- **Servizi per il codice sorgente:** Gestiscono l'elaborazione e la formattazione dei file sorgente.
- **Configurazione e file system:** Si occupano rispettivamente della lettura e validazione delle configurazioni e dell'accesso al file system.

Il modello di comunicazione ibrido permette di sfruttare la flessibilità degli eventi di $VSCode_{[G]}$ nella comunicazione tra presenter e view mantenendo al contempo un flusso di controllo chiaro e prevedibile all'interno del model.

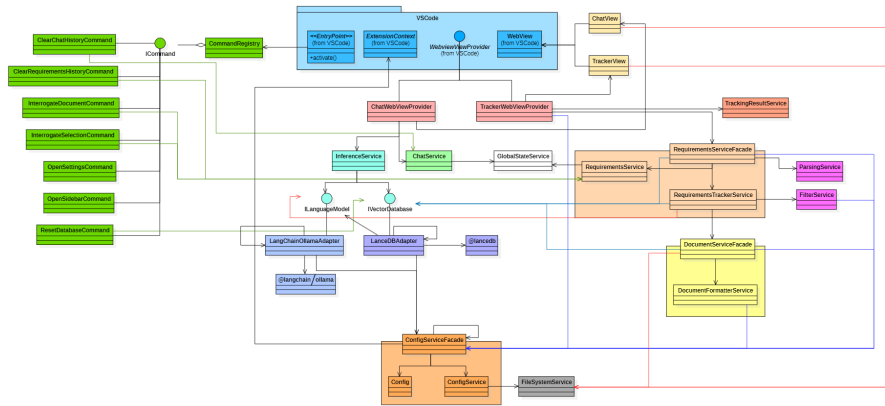


Figure 1: UML Diagram

3.4.1 Architettura di dettaglio

3.4.2 Model

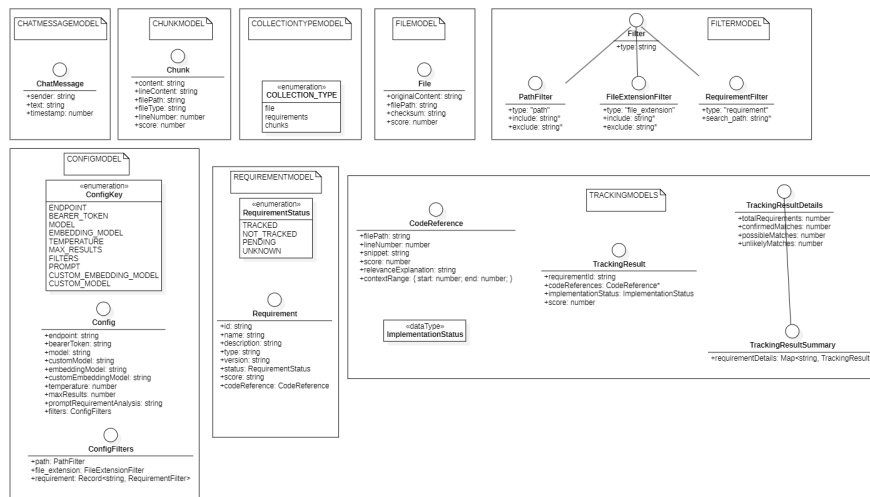


Figure 2: Models UML Diagram

3.4.2.1 ChatMessage

L'interfaccia *ChatMessage* ha lo scopo di modellare un messaggio di una chat. Definisce la struttura di ogni messaggio indicando il mittente (*sender*) che può essere l'utente (*user*) o il modello $LLM_{[G]}$ (*model*), il contenuto del messaggio in forma di stringa (*text*) e un timestamp che indica quando il messaggio è stato inviato.

3.4.2.2 File L'interfaccia *File* ha lo scopo di modellare un file all'interno del progetto. Memorizza all'interno di campi testuali: il contenuto originale del file (*originalContent*), il percorso (*filepath*), il checksum per la verifica dell'integrità (*checksum*) e come campo numerico opzionale, un punteggio usato per valutare la rilevanza (*score*).

3.4.2.3 Requirement L'interfaccia *Requirement* ha lo scopo di modellare un *requisito software*_[G]. Memorizza, all'interno di campi testuali: l'identificativo univoco (*_id*), il nome (*name*), la descrizione (*description*), la tipologia (*type*) e la versione (*version*). Inoltre vengono memorizzati: lo stato tramite un oggetto del tipo *RequirementStatus* (*status*) e come campi opzionali il punteggio (*score*) e la porzione di codice dove è stato implementato (*codereference*). *RequirementStatus* è un enum definito all'interno che rappresenta lo stato di implementazione di un requisito: tracciato (*TRACKED*), non tracciato (*NOT_TRACKED*), pendente (*PENDING*) e sconosciuto (*UNKNOWN*).

3.4.2.4 Chunk L'interfaccia *Chunk* ha lo scopo di modellare un frammento di codice o testo a partire da un file. Memorizza in campi testuali il contesto (*content*), il contenuto della linea (*lineContent*), il percorso del file di partenza (*filePath*) e la tipologia di file (*fileType*), mentre in campi numerici memorizza la lunghezza in linee (*lineNumber*) e, come opzionale, un punteggio usato per valutare la rilevanza (*score*).

3.4.2.5 CollectionType L'enum *CollectionType* ha lo scopo di modellare di categorizzare i tipi di collezioni disponibili nel sistema: file (*file*), requisiti (*requirements*) e frammenti di codice (*chunks*).

3.4.2.6 Config Il modulo *Config* ha lo scopo di modellare la configurazione del sistema. Contiene un enum (*ConfigKey*) e la rispettiva interfaccia (*Config*) la quale rappresenta l'insieme delle configurazioni. In campi testuali vengono rappresentati: endpoint dell'applicativo *Ollama*_[G] (*endpoint*), il token di autenticazione (*bearerToken*), il nome del modello per la generazione di codice (*model*), il nome del modello per l'embedding (*embeddingModel*) e il prompt personalizzato (*promptRequirementAnalysis*). Come opzionali, sono disponibili: un campo la selezione di un modello di embedding personalizzato (*customEmbeddingModel*) e un campo per la selezione di un modello per la generazione di codice personalizzato (*customModel*). In campi numerici vengono rappresentati la temperatura dei modelli (*temperature*) e il numero massimo di risultati (*maxResults*). Inoltre è presente un campo per l'interfaccia *ConfigFilters* anch'essa definita all'interno del modulo e che rappresenta i filtri applicabili nella ricerca: basati su percorsi dei file (*path*), sulle estensioni dei file (*file_extension*) e sui requisiti selezionati (*requirement*).

3.4.2.7 Filter L'interfaccia *Filter* ha lo scopo di modellare un filtro. L'interfaccia è estesa dalle interfacce: *PathFilter* che rappresenta un filtro per percorso file, *FileExtensionFilter* che rappresenta un filtro per tipologia di estensione del file e *RequirementFilter* che rappresenta un filtro per requisito.

3.4.2.8 TrackingModel Il modulo *TrackingModel* ha lo scopo di modellare il tracciamento dell'implementazione dei requisiti all'interno dei file selezionati. Definisce l'interfaccia *CodeReference* che rappresenta il riferimento ad una porzione di codice. Contiene, come valore testuale, il percorso del file (*filePath*), il frammento effettivo del codice (*snippet*) e come opzionale il motivo della scelta dell'accoppiamento (*relevanceExplanation*). In forma numerica, specifica le linee di codice (*lineNumber*) e il punteggio (*score*). Definisce l'interfaccia *TrackingResult* che associa un requisito (*requirementId*) con una porzione di codice (*codeReference*), il punteggio (*score*) e lo stato di implementazione. Contiene, inoltre, un oggetto del tipo *ContextRange* che contiene la linea iniziale e la linea finale dello *snippet*. Definisce l'interfaccia *TrackingResultDetails* che rappresenta lo stato globale del tracciamento dei requisiti. Contiene, come valori numerici, il totale dei requisiti (*totalRequirements*), il numero dei requisiti confermati (*confirmedMatches*), il numero di possibili match codice-requisito (*possibleMatches*) e match improbabili (*unlikelyMatches*). Definisce l'interfaccia *TrackingResultSummary* che estende *TrackingResultDetails* aggiungendo una mappa con oggetti del tipo *TrackingResult* e il requisito.

- **handleMessageFromWebview** il quale permette di elaborare i messaggi ricevuti dalla webview e indirizza ai metodi specifici in base al tipo.
- **onSendMessage** (*privato*) il quale permette di gestire l'invio di un messaggio dell'utente, ottenendo e mostrando una risposta dal modello.
- **onClearHistory** (*privato*) il quale permette di cancellare la cronologia dei messaggi della chat.
- **sendMessageToWebview** (*privato*) il quale permette di inviare un messaggio alla webview per aggiornare l'interfaccia utente.

3.4.4.2 TrackerWebViewProvider La classe *TrackerWebViewProvider* è responsabile della gestione della *TrackerWebView* che rappresenta il tracciamento dei requisiti. Mette a disposizione i metodi:

- **resolveWebviewView** il quale permette di configurare e inizializzare la vista webview quando viene creata.
- **onChangeTextEditorSelection** il quale permette di gestire gli eventi di selezione del testo nell'editor quando in modalità di modifica.
- **onAnalyzeImplementation** (*privato*) il quale permette di analizzare l'implementazione di un requisito rispetto ai riferimenti di codice.
- **webviewViewConfiguration** (*privato*) il quale permette di configurare le opzioni della webview e imposta l'HTML iniziale.
- **webviewViewHandleEvents** (*privato*) il quale permette di registrare i gestori degli eventi per i messaggi ricevuti dalla webview.
- **sendMessageToWebview** (*privato*) il quale permette di inviare un messaggio alla webview per aggiornare l'interfaccia utente.
- **handleMessageFromWebview** (*privato*) il quale permette di elaborare i messaggi ricevuti dalla webview e indirizza ai metodi specifici.
- **onTabToImport** (*privato*) il quale permette di attivare la scheda di importazione dei requisiti.
- **onTabToTrack** (*privato*) il quale permette di attivare la scheda di tracciamento dei requisiti.
- **onTabToResults** (*privato*) il quale permette attivare la scheda dei risultati del tracciamento.
- **onCancelEditImplementation** (*privato*) il quale permette di annullare la modifica dell'implementazione di un requisito.
- **onConfirmEditImplementation** (*privato*) il quale permette di confermare la modifica dell'implementazione di un requisito.
- **onStartEditMode** (*privato*) il quale permette di avviare la modalità di modifica per un riferimento di codice di un requisito.
- **startEditMode** (*privato*) il quale permette di implementare la logica di avvio della modalità di modifica.
- **onEndEditMode** (*privato*) il quale permette di terminare la modalità di modifica.
- **onRejectRequirementImplementation** (*privato*) il quale permette di rifiutare un'implementazione di requisito specifica.
- **onConfirmRequirementImplementation** (*privato*) il quale permette di confermare un'implementazione di requisito specifica.
- **onImportRequirements** (*privato*) il quale permette di importare requisiti da un file con formato specifico.
- **onTrackRequirements** (*privato*) il quale permette di avviare il tracciamento dei requisiti specificati nel codice.
- **onOpenFile** (*privato*) il quale permette di aprire un file specifico nell'editor e si posiziona alla riga indicata.

- **onClearRequirements** (*privato*) il quale permette di cancellare tutti i requisiti attualmente tracciati.
- **updateRequirementsDisplay** (*privato*) il quale permette di aggiornare la visualizzazione dei requisiti nella webview.
- **updateTrackingResultsDisplay** (*privato*) il quale permette di aggiornare la visualizzazione dei risultati di tracciamento nella webview.
- **onEditRequirement** (*privato*) il quale permette di avviare la modifica di un requisito specifico.
- **onDeleteRequirement** (*privato*) il quale permette di eliminare un requisito specifico.
- **stopEditMode** (*privato*) il quale permette di arrestare la modalità di modifica attuale e ripristina lo stato normale.
- **serializeTrackingResults** (*privato*) il quale converte in formato leggibile dalla view un oggetto del tipo *TrackingResultSummary*.

3.4.5 Command

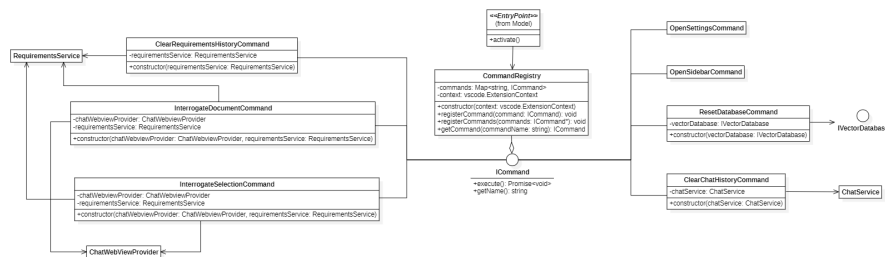


Figure 5: Commands UML Diagram

3.4.5.1 CommandsRegistry La classe *CommandsRegistry* si occupa di gestire la registrazione e l'esecuzione dei comandi dell'estensione (*ICommand*). Infatti è responsabile della gestione tra l'interfaccia utente e la business logic. Mette a disposizione i metodi:

- **registerCommand** il quale registra un singolo comando nell'estensione e lo aggiunge al registro interno. Il comando viene collegato alla sua funzione di esecuzione e la sottoscrizione viene aggiunta al contesto dell'estensione.
- **registerCommands** il quale registra più comandi contemporaneamente, utilizzando il metodo *registerCommand* per ciascuno.
- **getCommand** il quale restituisce un comando specifico dal registro in base al nome.

I comandi implementati nell'estensione sono classi che estendono *ICommand*. Tutti mettono a disposizione i metodi *getName*, che restituisce il nome del comando, e *execute* contenente l'effettivo comportamento. Le classi sono:

- **ClearChatHistoryCommand** permette la cancellazione della cronologia delle conversazioni della chat. È accessibile con il nome "requirementsTracker.clearChatHistory".
- **ClearRequirementsHistoryCommand** permette la cancellazione di tutti i requisiti caricati nel sistema. È accessibile con il nome "requirementsTracker.clearRequirementsHistory".
- **InterrogateDocumentCommand** permette di analizzare l'intero contenuto del documento attivo nel contesto dei requisiti caricati. È accessibile con il nome "requirementsTracker.interrogateDocument".
- **InterrogateSelectionCommand** permette di analizzare solo il testo selezionato nell'editor attivo nel contesto dei requisiti caricati. È accessibile con il nome "requirementsTracker.interrogateSelection".

- **OpenSettingsCommand** permette di aprire direttamente la pagina delle impostazioni dell'estensione Requirements Tracker nell'interfaccia delle impostazioni di *VSCode_[G]*. È accessibile con il nome "requirementsTracker.openSettings".
- **OpenSidebarCommand** permette di aprire la barra laterale dell'estensione Requirements Tracker all'interno di *VSCode_[G]*. È accessibile con il nome "requirementsTracker.openSidebar".
- **ResetDatabaseCommand** permette di reimpostare il database vettoriale utilizzato per l'indicizzazione di codice e requisiti. È accessibile con il nome "requirementsTracker.resetDatabase".

3.4.5.2 ICommand L'interfaccia *ICommand* ha lo scopo di modellare un comando dell'applicazione. Modella i metodi:

- **execute** il quale permette l'esecuzione del comando e restituisce un *Promise*.
- **getName** il quale permette la restituzione del nome del comando.

3.4.6 Servizi e facade

3.4.6.1 FileSystemService

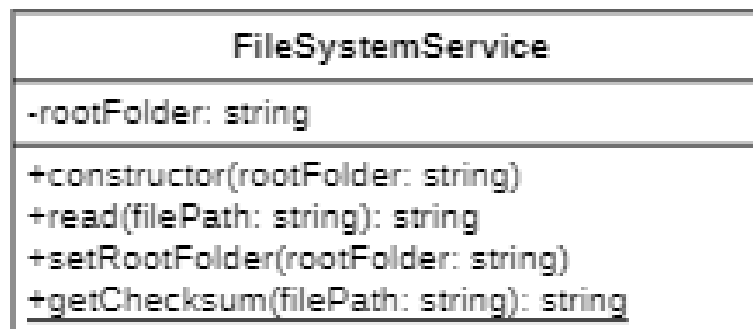


Figure 6: FileSystem Service UML Diagram

La classe *FileSystemService* si occupa delle interazioni con il file system per la gestione dei file. Mette a disposizione i metodi:

- **read** il quale permette la lettura di un file a partire dal suo percorso.
- **setRootFolder** il quale permette di indicare la cartella di root.
- **getChecksum** il quale calcola il checksum di un file a partire dal suo percorso.

3.4.6.2 GlobalStateService

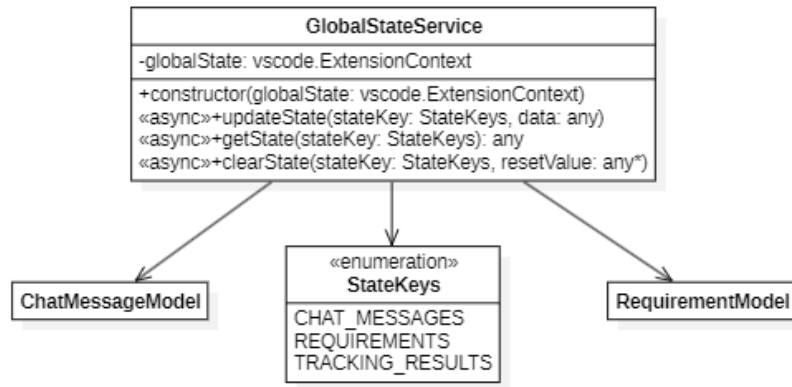


Figure 7: GlobalState Service UML Diagram

La classe *GlobalStateService* si occupa della gestione dello stato globale dell'estensione usando l'API di stato globale di *VSCode_[G]*. Mette a disposizione i metodi:

- **updateState** il quale permette di aggiornare lo stato dell'applicazione riguardo i requisiti o la chat.
- **getState** il quale permette di ottenere lo stato dell'applicazione riguardo i requisiti o la chat.
- **clearState** il quale permette di cancellare lo stato dell'applicazione riguardo i requisiti o la chat.

3.4.6.3 Config

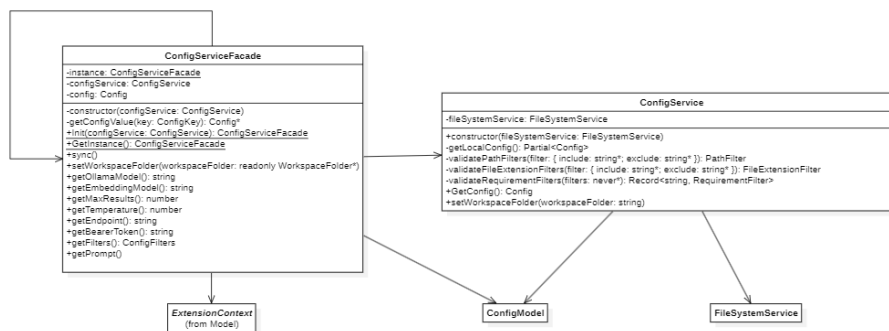


Figure 8: Config Service UML Diagram

3.4.6.3.1 ConfigService La classe *ConfigService* si occupa di gestire la configurazione dell'estensione combinando le impostazioni globali provenienti da *VSCode_[G]* con le configurazioni del progetto. Al suo interno contiene un'istanza del servizio *FileSystemService* per l'aggiornamento delle configurazioni dell'*applicativo_[G]*. Mette a disposizione i metodi:

- **GetConfig** il quale ritorna un array di oggetti contenenti lo stato di tutte le configurazioni.
- **setWorkspaceFolder** il quale imposta la root folder del filesystem.
- **getLocalConfig** (*privato*) il quale ritorna lo stato delle configurazioni locali.
- **validatePathFilters** (*privato*) il quale controlla la validità dei percorsi indicati nei filtri.
- **validateFileExtensionFilters** (*privato*) il quale controlla la validità delle estensioni indicate nei filtri.
- **validateRequirementFilters** (*privato*) il quale controlla la validità dei requisiti indicati nei filtri.

3.4.6.3.2 ConfigServiceFacade La classe *ConfigServiceFacade* fornisce un accesso semplificato alla configurazione del sistema. Implementa il pattern facade e utilizza un singleton per garantire un'unica istanza dell'applicazione. Mette a disposizione i metodi:

- **Init** (*statico*) il quale inizializza l'istanza singleton e la restituisce.
- **GetInstance** (*statico*) il quale restituisce l'istanza singleton esistente.
- **sync** il quale sincronizza la cache interna con le configurazioni più recenti.
- **getConfigValue** (*privato*) il quale restituisce un valore di configurazione specifico, sincronizzando se necessario.
- **getOllamaModel** il quale restituisce il nome del modello per la generazione del codice scelto.
- **getEmbeddingModel** il quale restituisce il modello di embedding scelto.
- **getMaxResults** il quale restituisce il numero massimo di risultati configurato.
- **getTemperature** il quale restituisce la temperatura configurata per il modello.
- **getEndpoint** il quale restituisce l'endpoint di *Ollama_[G]* configurato.
- **getBearerToken** il quale restituisce il token di autenticazione configurato.
- **getFilters** il quale restituisce i filtri scelti.
- **getPrompt** il quale restituisce il prompt scelti.
- **setWorkspaceFolder** il quale imposta la root folder del filesystem.

3.4.6.4 Requirement

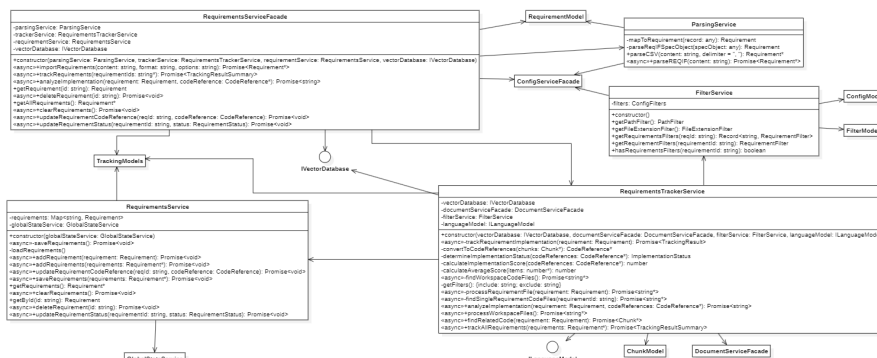


Figure 9: Requirement Service UML Diagram

3.4.6.4.1 RequirementsService La classe *RequirementsService* si occupa della gestione dei requisiti mappati con identificativo e oggetto *Requirement* e interagisce con il *GlobalStateService*. Mette a disposizione i metodi:

- **addRequirement** i quali permettono l'aggiunta di un singolo requisito.
- **addRequirements** i quali permettono l'aggiunta di più requisiti.
- **updateRequirementCodeReference** il quale si occupa di aggiornare la posizione del codice che implementa ciascun requisito.
- **updateRequirementStatus** il quale si occupa di aggiornare lo status di ciascun requisito.
- **saveRequirements** il quale memorizza i requisiti.
- **getRequirements** il quale restituisce la lista dei requisiti.
- **clearRequirements** il quale cancella la lista dei requisiti.
- **getById** il quale restituisce un requisito a partire dal suo id.

- **deleteRequirement** il quale cancella un requisito a partire dal suo id.
- **saveRequirements** (*privato*) il quale salva nel *GlobalStateService* i requisiti.
- **loadRequirements** (*privato*) il quale aggiorna nel *GlobalStateService* i requisiti.

3.4.6.4.2 RequirementsServiceFacade La classe *RequirementsServiceFacade* fornisce un accesso semplificato al servizio di tracciamento dei requisiti (*RequirementsTrackerService*, *ParsingService* e *RequirementsService*). Mette a disposizione i metodi:

- **importRequirements** il quale importa requisiti da file csv o reqif, li salva e crea embeddings.
- **trackRequirements** il quale traccia l'implementazione dei requisiti specificati.
- **analyzeImplementation** il quale analizza se un riferimento al codice implementa un requisito specifico.
- **getRequirement** il quale restituisce un requisito specifico per id.
- **deleteRequirement** il quale elimina un requisito specifico per id.
- **getAllRequirements** il quale restituisce tutti i requisiti.
- **clearRequirements** il quale elimina tutti i requisiti.
- **updateRequirementCodeReference** il quale si occupa di aggiornare la posizione del codice che implementa ciascun requisito.
- **updateRequirementStatus** il quale si occupa di aggiornare lo status di ciascun requisito.

3.4.6.4.3 ParsingService La classe *ParsingService* si occupa dell'analisi e della conversione di file in formato csv o reqif in una struttura dati contenente i requisiti memorizzati al loro interno. Mette a disposizione i metodi:

- **parseCSV** il quale gestisce la conversione dei file in formato csv che utilizzano separatori differenti.
- **parseREQIF** il quale gestisce la conversione dei file in formato reqif.
- **mapToRequirement** (*privato*) il quale restituisce un oggetto del tipo *Requirement* a partire dai singoli record ottenuti dal file CSV.
- **parseReqIFObject** (*privato*) il quale restituisce un oggetto del tipo *Requirement* a partire dai singoli record ottenuti dal file reqif.

3.4.6.4.4 FilterService La classe *FilterService* si occupa della gestione dei filtri ottenuti tramite *ConfigServiceFacade* e memorizzati in un oggetto *ConfigFilters*. Mette a disposizione i metodi:

- **getPathFilter** il quale restituisce i filtri per path memorizzati all'interno dell'istanza di *ConfigFilters*.
- **getFileExtensionFilter** il quale restituisce i filtri per estensione del file memorizzati all'interno dell'istanza di *ConfigFilters*.
- **getRequirementsFilter** il quale restituisce i filtri per tutti i requisiti memorizzati all'interno dell'istanza di *ConfigFilters*.
- **getRequirementFilter** il quale restituisce il filtro per il requisito specificato memorizzato all'interno dell'istanza di *ConfigFilters*.
- **hasRequirementsFilter** il quale restituisce se è presente un filtro, per il requisito specificato, memorizzato all'interno dell'istanza di *ConfigFilters*.

3.4.6.4.5 RequirementsTrackerService La classe *RequirementsTrackerService* si occupa della gestione del tracciamento dell'implementazione dei requisiti nel codice sorgente. Interagisce con il database vettoriale (*IVectorDatabase*), il servizio di gestione dei documenti (*DocumentServiceFacade*), i filtri (*FilterService*) e il modello LLM_[G] (*ILanguageModel*). Mette a disposizione i metodi:

- **analyzeImplementation** il quale analizza se il codice fornito implementa effettivamente il requisito creando il prompt con il quale interrogare il modello.
- **trackRequirementImplementation** il quale traccia l'implementazione di un singolo requisito, trovando codice correlato, convertendolo in riferimenti e determinando lo stato di implementazione e il punteggio.
- **processWorkspaceFiles** il quale elabora tutti i file del workspace applicando filtri, tramite il *DocumentServiceFacade* e restituendo l'elenco dei file elaborati.
- **findRelatedCode** il quale cerca nel database vettoriale porzioni di codice correlate al requisito specificato a partire dalla descrizione del requisito.
- **convertToCodeReferences** (*privato*) il quale converte i frammenti di codice (chunks) in riferimenti al codice strutturati, ordinati per punteggio di rilevanza.
- **trackAllRequirements** il quale traccia tutti i requisiti forniti e restituisce un riepilogo complessivo.
- **determineImplementationStatus** (*privato*) il quale determina lo stato di implementazione in base al punteggio medio dei riferimenti al codice.
- **calculateImplementationScore** (*privato*) il quale calcola un punteggio complessivo per l'implementazione basato sui singoli riferimenti al codice trovati.
- **calculateAverageScore** (*privato*) il quale calcola il punteggio medio del punteggio dell'implementazione.
- **findWorkspaceCodeFiles** (*privato*) il quale trova tutti i file di codice nel workspace *VSCode_[G]* applicando i filtri di inclusione ed esclusione configurati.
- **getFilters** (*privato*) il quale restituisce i filtri per file e percorsi.
- **processRequirementFile** (*privato*) il quale elabora i file associati al requisito applicando filtri.
- **findSingleRequirementCodeFiles** (*privato*) il quale trova i file associati al requisito applicando i filtri di inclusione ed esclusione configurati.

3.4.6.5 TrackingResultService

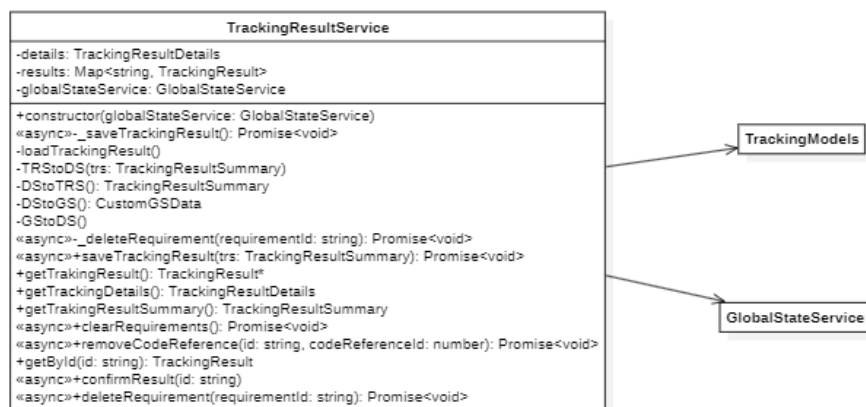


Figure 10: Tracking Result Service UML Diagram

La classe *TrackingResultService* si occupa della gestione della persistenza e della manipolazione dei risultati del tracciamento dei requisiti. Mette a disposizione i metodi:

- **saveTrackingResult** il quale salva i risultati del tracciamento, convertendoli nel formato di archiviazione interno e memorizzandoli nello stato globale.
- **getTrackingResult** il quale restituisce tutti i risultati di tracciamento come un array di oggetti *TrackingResult*.

- **getTrackingDetails** il quale restituisce i dettagli di riepilogo del tracciamento.
- **getTrakingResultSummary** il quale converte i dati in un oggetto `TrackingResultSummary` e lo restituisce.
- **clearRequirements** il quale elimina tutti i dati di tracciamento sia dalla memoria che dallo stato globale.
- **deleteRequirement** il quale elimina i dati di tracciamento di un singolo requisito sia dalla memoria che dallo stato globale.
- **removeCodeReference** il quale rimuove uno specifico riferimento al codice dal requisito tracciato, aggiornando i contatori e persistendo le modifiche.
- **getById** il quale restituisce un singolo risultato di tracciamento in base all'id del requisito.
- **confirmResult** il quale marca un risultato di tracciamento come confermato, aggiornando i contatori e rimuovendolo da quelli in stato pending.
- **deleteRequirement** (*privato*) il quale elimina i dati di tracciamento di un singolo requisito sia dalla memoria che dallo stato globale.
- **saveTrackingResult** (*privato*) il quale memorizza i dati di tracciamento nello stato globale.
- **loadTrackingResult** (*privato*) il quale aggiorna i dati di tracciamento nello stato globale.
- **TRStoDS** (*privato*) il quale converte un oggetto `TrackingResultSummary` nelle strutture dati interne del servizio.
- **DStoTRS** (*privato*) il quale converte le strutture dati interne in un oggetto `TrackingResultSummary`.
- **DStoGS** (*privato*) il quale converte le strutture dati interne nel formato richiesto per la persistenza nello stato globale.
- **GStoDS** (*privato*) il quale converte i dati provenienti dallo stato globale nelle strutture dati interne del servizio.

3.4.6.6 Document

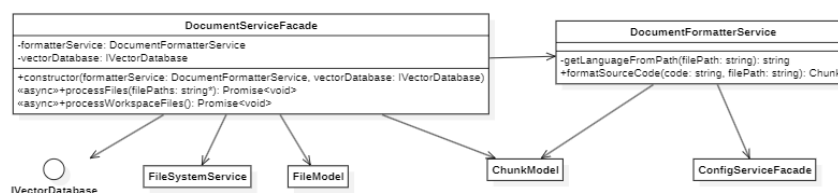


Figure 11: Document Service UML Diagram

3.4.6.6.1 DocumentFormatterService La classe `DocumentFormatterService` si occupa della formattazione del codice sorgente in frammenti (*chunks*) a seconda del linguaggio rilevato dal path (*language*). Mette a disposizione il metodo:

- **getLanguageFromPath** (*privato*) il quale restituisce il linguaggio a partire dall'estensione del file.
- **formatSourceCode** il quale suddivide il file in chunks.

3.4.6.6.2 DocumentServiceFacade La classe `DocumentServiceFacade` fornisce un accesso semplificato al servizio di gestione dei documenti (`DocumentFormatterService`). Mette a disposizione i metodi:

- **processFiles** il quale elabora un elenco di file, formattandoli e aggiungendoli al database vettoriale. Gestisce controlli di dimensione, calcolo di checksum e verifica dell'esistenza.
- **processWorkspaceFiles** il quale trova e processa tutti i file presenti nella cartella del progetto.

3.4.6.7 Inference

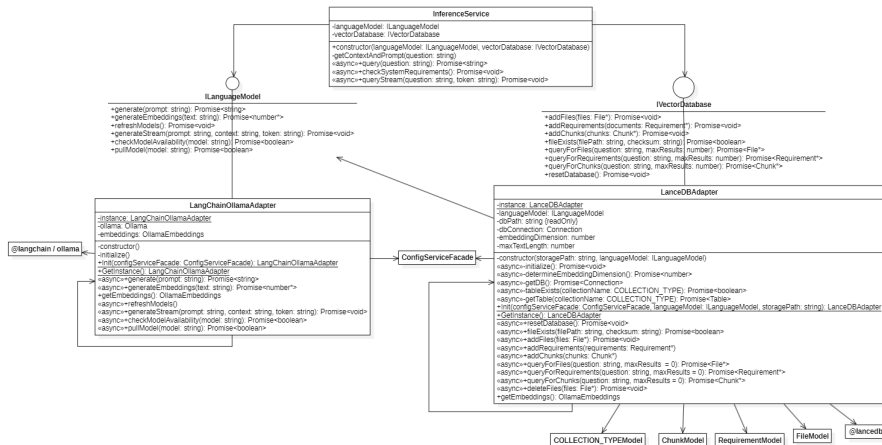


Figure 12: Inference Service UML Diagram

3.4.6.7.1 InferenceService La classe *InferenceService* si occupa della gestione delle inference basate sui modelli $LLM_{[G]}$ a partire da query eseguite sul database vettoriale. Mette a disposizione i metodi:

- **query** il quale permette di effettuare una chiamata a $LLM_{[G]}$ tramite $Ollama_{[G]}$.
- **checkSystemRequirement** il quale permette di effettuare un controllo sulla connessione al servizio $Ollama_{[G]}$.
- **queryStream** il quale permette di effettuare una chiamata a $LLM_{[G]}$ tramite $Ollama_{[G]}$ fornendo il risultato come stream.
- **getContextEndPrompt** (*privato*) il quale genera il prompt per l'interrogazione al modello a partire dal contesto.

3.4.6.7.2 ILanguageModel L'interfaccia *ILanguageModel* ha lo scopo di modellare l'interazione con i modelli $LLM_{[G]}$. Modella i metodi:

- **generate** il quale genera una risposta testuale a partire da un prompt.
- **generateEmbeddings** il quale esegue l'embedding di una stringa.
- **refreshModels** il quale aggiorna i modelli in seguito a cambiamenti nella configurazione.
- **generateStream** il quale genera una risposta sotto forma di stream a partire da un prompt.
- **checkModelAvailability** il quale controlla se il modello è disponibile nell'endpoint indicato.
- **pullModel** il quale scarica il modello nell'endpoint indicato.

3.4.6.7.3 IVectorDatabase L'interfaccia *IVectorDatabase* ha lo scopo di modellare l'interazione con il database vettoriale. Modella i metodi:

- **addFiles** il quale permette l'aggiunta di un file nel database vettoriale.
- **addRequirements** il quale permette l'aggiunta di un requisiti nel database vettoriale.
- **addChunks** il quale permette l'aggiunta di frammenti di codice nel database vettoriale.
- **fileExists** il quale verifica se un file con un dato percorso e checksum esiste già nel database.
- **queryForFiles** il quale cerca file simili semanticamente alla domanda fornita.
- **queryForRequirements** il quale cerca requisiti simili semanticamente alla domanda fornita.

- **queryForChunks** il quale cerca frammenti di codice simili semanticamente alla domanda fornita.
- **resetDatabase** il quale permette di reimpostare il database, eliminando tutti i dati memorizzati.

3.4.6.7.4 LangChainOllamaAdapter La classe *LangChainOllamaAdapter* implementa l'interfaccia *ILanguageModel* e si occupa di fornire un'implementazione per l'interazione dei modelli usando *Ollama_[G]*. Oltre a fornire l'implementazione dei metodi dell'interfaccia, mette a disposizione:

- **Init** (*statico*) il quale inizializza l'istanza singleton e la restituisce.
- **GetInstance** (*statico*) il quale restituisce l'istanza singleton esistente.
- **initialize** (*privato*) il quale permette di configurare le istanze di *Ollama_[G]* e *OllamaEmbeddings* con i parametri di configurazione.
- **getEmbeddings** il quale permette di restituire l'istanza di *OllamaEmbeddings* utilizzata per generare gli embedding.
- **generate** il quale genera una risposta testuale a partire da un prompt.
- **generateEmbeddings** il quale esegue l'embedding di una stringa.
- **refreshModels** il quale aggiorna i modelli in seguito a cambiamenti nella configurazione.
- **generateStream** il quale genera una risposta sotto forma di stream a partire da un prompt.
- **checkModelAvailability** il quale controlla se il modello è disponibile nell'endpoint indicato.
- **pullModel** il quale scarica il modello nell'endpoint indicato.

3.4.6.7.5 LanceDBAdapter La classe *LanceDbAdapter* implementa l'interfaccia *IVectorDatabase* e si occupa di fornire un'implementazione per la gestione dei file e dei requisiti usando LanceDB come database vettoriale. Oltre a fornire l'implementazione dei metodi dell'interfaccia, mette a disposizione:

- **Init** (*statico*) il quale inizializza l'istanza singleton e la restituisce.
- **GetInstance** (*statico*) il quale restituisce l'istanza singleton esistente.
- **deleteFiles** il quale permette di eliminare file dal database.
- **getEmbeddings** il quale restituisce l'istanza di *OllamaEmbeddings* utilizzata per generare gli embedding.
- **addFiles** il quale permette l'aggiunta di un file nel database vettoriale.
- **addRequirements** il quale permette l'aggiunta di un requisiti nel database vettoriale.
- **addChunks** il quale permette l'aggiunta di frammenti di codice nel database vettoriale.
- **fileExists** il quale verifica se un file con un dato percorso e checksum esiste già nel database.
- **queryForFiles** il quale cerca file simili semanticamente alla domanda fornita.
- **queryForRequirements** il quale cerca requisiti simili semanticamente alla domanda fornita.
- **queryForChunks** il quale cerca frammenti di codice simili semanticamente alla domanda fornita.
- **resetDatabase** il quale permette di reimpostare il database, eliminando tutti i dati memorizzati.
- **initialize** (*privato*) il quale permette di inizializzare la connessione al database e configura il servizio di embedding.
- **determineEmbeddingDimension** (*privato*) il quale permette di determinare la dimensione degli embedding generando un embedding di prova.
- **getDB** (*privato*) il quale permette di ottenere o creare una connessione al database.
- **tableExists** (*privato*) il quale permette di verificare se una tabella esiste nel database.
- **getTable** (*privato*) il quale permette di ottenere o creare una tabella nel database in base al tipo di collezione.

3.4.6.8 ChatService

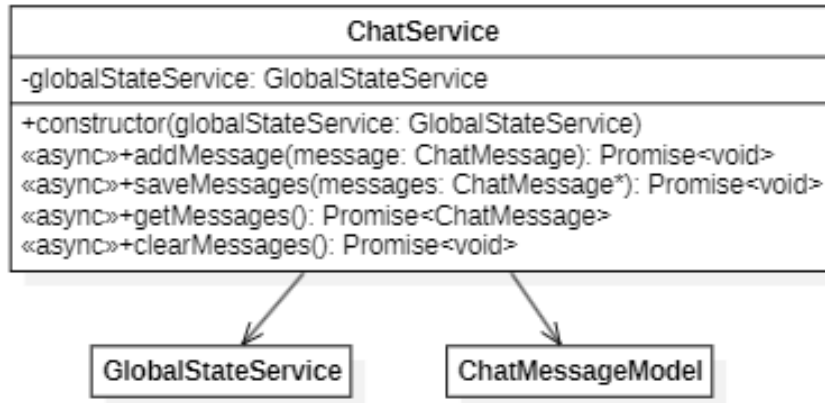


Figure 13: Chat Service UML Diagram

La classe *ChatService* rappresenta il servizio di chat fornito dall'estensione. Al suo interno contiene un'istanza del servizio *GlobalStateService* per l'aggiornamento dello stato dell'applicativo_[G]. Mette a disposizione i metodi asincroni:

- **addMessage** il quale permette l'aggiunta di nuovi messaggi.
- **saveMessage** il quale permette il salvataggio della cronologia della chat.
- **getMessage** il quale permette di recuperare un particolare messaggio in arrivo.
- **clearMessage** il quale permette di cancellare la cronologia della chat.

3.5 Progettazione grafica

L'estensione si integra direttamente nell'interfaccia di *VSCode*_[G], aggiungendo un nuovo pannello laterale identificato dal nome "REQUIREMENTS TRACKER". L'icona del pannello, rappresentata dal logo di *Ollama*_[G], è visibile nella barra laterale ("barra attività") solitamente posizionata a sinistra.

L'estensione implementa due pannelli:

- **Requirements** che a sua volta contiene i pannelli **Import**, **Tracker** e **Results**
- **Chat**

3.5.1 Import

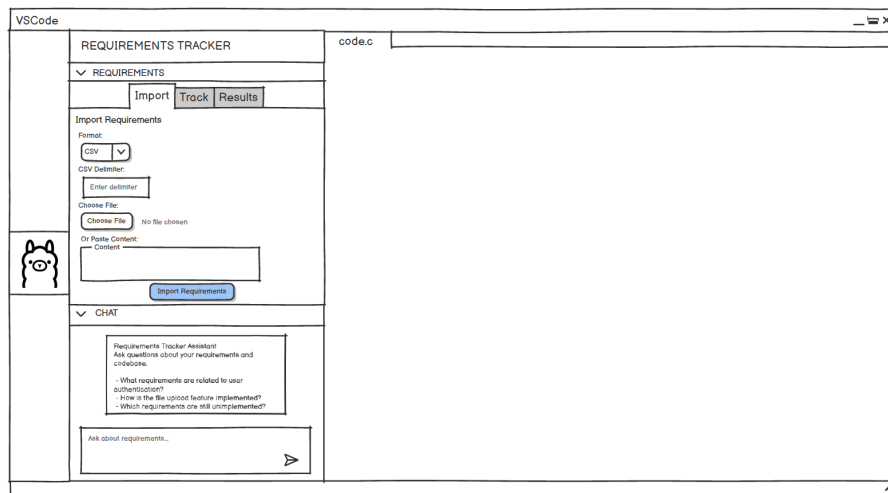


Figure 14: Pannello Import

Il pannello *Import* permette di caricare dei requisiti in due modalità:

- Caricamento di un file locale (es. .csv o .reqIF), selezionabile tramite il pulsante “Choose File”.
- Incollando i requisiti direttamente sulla sezione apposita (sotto “Or Paste Content”).

Per caricare i requisiti bisogna inoltre selezionare il tipo di file (CSV o reqIF) sul pulsante di selezione sotto la sezione “Format”. Esiste anche l'opzione di scegliere un delimitatore CSV personalizzato sotto la sezione “CSV Delimiter”. Una volta scelto il file o dopo aver incollato a mano i requisiti, questi possono essere caricati tramite il pulsante “Import Requirements”.

3.5.2 Track

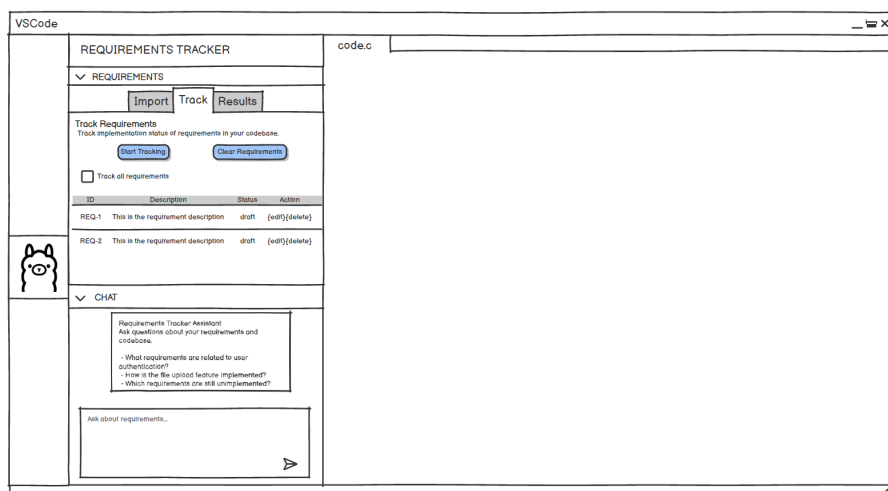


Figure 15: Pannello Track

Nel pannello *Track* l'utente può:

- Tracciare requisiti specifici selezionandoli dalla lista caricata.
- Tracciare tutti i requisiti contemporaneamente selezionando l'opzione "Track all requirements".

Il tracciamento si avvia con il pulsante "Start Tracking".

È inoltre disponibile la funzionalità "Clear Requirements" che rimuove tutti i requisiti attualmente caricati.

3.5.3 Results

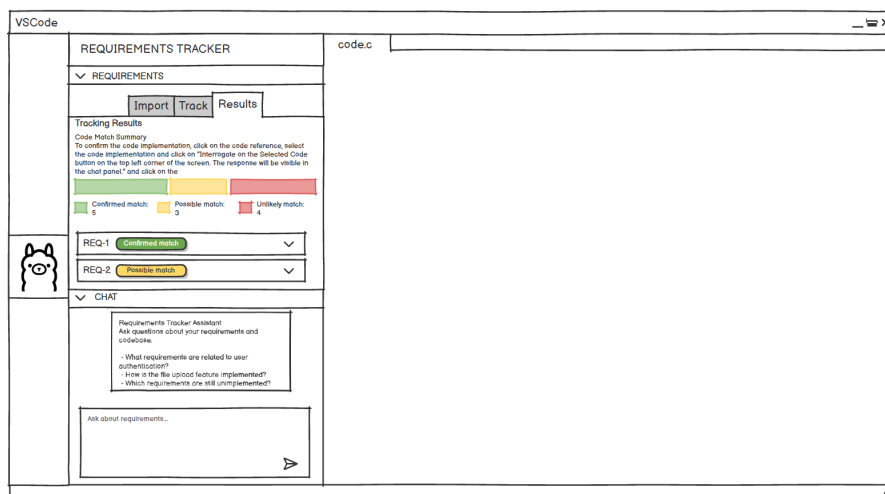


Figure 16: Pannello Results

Nel pannello Results vengono mostrati i risultati dell'analisi:

- Una sezione "Code Match Summary" presenta un riepilogo dei requisiti suddivisi in Confirmed match, Possible match e Unlikely match.
- L'utente può visualizzare i dettagli di ciascun requisito tracciato e cliccare su un riferimento per confermare o modificare l'implementazione proposta.

3.5.4 Chat

Il pannello Chat consente all'utente di porre domande sui requisiti e sulla loro implementazione. L'assistenza è fornita tramite modelli di intelligenza artificiale integrati con *Ollama*_[G], e può essere utilizzata per approfondimenti, verifiche o spiegazioni specifiche relative al contenuto e allo stato dei requisiti.